

eboard readme file, devbuild 2026-01-12:

for license information see: license.md or source files, most if not all should contain a license information. The license information displayed in the about plugin is the most recent and should be used.

project team: contact via email: kammel@posteo.de

Project description:

Eboard is a prototype and serves as an experimental platform for research and development of fluid user interfaces, developed using MVVM pattern and C#/WPF.

Project questions:

How does the ability to change a variety of properties at once in a very limited amount of time affect stress, creativity, concentration, motivation and workflow?

What are the minimum required steps per task? What is the maximum of features needed for supporting personal or small to minor art, educational, administrative or business projects?

Project concept and history:

Since the beginnings in 2022 the goal is to build user interfaces that allow its users to change a variety of context areas to the users liking, without limiting the complexity of the features encapsulated, while also being easy to look at and fast to work with. The term fluid ui is used since the end of 2025 in the prototype to encapsulate the features that allow changes to context areas and to separate those from other features. Eboard is a successor to AEUI (2024, Aiding Elements User Interface) and YSUI (2022, YRS, Your Startup UI), both written in C#, using WPF framework, aiming towards the same goal. Eboard follows several fresh approaches while aiming at better code and architectural and logic quality.

Project status 2026-01-13:

The prototype underwent some major changes to its codebase in the recent weeks. Although some parts have been refactored into fewer lines of code and better separation of concerns, many areas of the source are not yet refactored or robust. Some features have been added that will require further development and testing, but they showcase what they are about in an ok manner so far. Most if not all visible buttons should have the described effects and should work. Some menus use sliders and some of them need the hit of the nearby OK button to have effects.

The current state of the project is work in progress, there is no versioning besides dates. Some of its features are training exercises or experiments, others are hacked as mockups until better understanding or implementations are at hand or needed. The main goal is researching the concept itself, for now imagination must serve, where the proves of concept of the prototypes mockups ends.

Next steps will include further refactoring the source, improving memory usage, building better xaml styles (this requires some research into the topic and is low prio, because not that important at this stage), continuation of feature development and modularization of the project and plugin architecture, as well as basic drawing and command features, maybe even first mockup plugins for database access or server interaction.

Of minor priority are small improvements to existing features, fixing non critical bugs, code documentation and better integration of logging and exception handling into the code, fixing or adding xaml styles to ui elements like sliders, comboboxes and tabitems, which are still mostly on default style and are not yet affected by fluid ui changes.

Features:

→ **context areas:**

Mainwindow, browser, screens and elements are fluid ui context areas and will be explained further down below. Any other control or plugin can have context menus depending on the authors choices. Right clicking on any surface element will open a context menu. If the context is part of the fluid ui, it will contain a FluidUI submenu.

→ **fluid ui:**

fluid ui menus will differ a little bit depending on the context area and the function it serves. Any of them supports changes to the items 1-4 below:

- 1) Design: change of colors or even brushes for background, border, foreground and highlight. Highlight brush serves for indications of selection and for warnings to the user
- 2) Font: change of font family, font weight and font size
- 3) Size: change of margin, padding, border thickness, corner radius, width and height of the context areas outer border
- 4) Data: change of context title, description text, index text list, key text list and a total of 32 index and key text properties (the lists and 2x4xquads are experimental, the lists could be useful for internal logging activity or as a protocol/diary, the quads for riddles, hints or as storage mockup for potential properties like credentials, access or id keys, tokens and the like.)
- 5) Stand: if supported by the context area: change of position and orientation

→ **mainwindow context:**

the mainwindow context area is the outmost control and encapsulates all other context areas. It contains the plugin selection menu, the mainwindow buttons and toggle buttons, the screen display area and navigation buttons. It has no window caption. It can be leftclicked and dragged if the clicked surface does not contain clickable areas like buttons or scrollable areas and has an opacity or an alpha value(for brushes) above 0. Changes to its fluid ui will affect all of its controls, except for the other context areas. This can, given a distinguished design, help to identify main program functions fast and precise. It contains the following controls:

- >>> Eboard Title: center top textbox, shows the title of the mainwindow context area
- >>> Eboard Browser: togglebutton, keeps Eboard Browser context area visible or hidden.
- >>> Screen Control: togglebutton, keeps Screen Control ui visible or hidden, screen control ui is defined by the active screen context area fluid ui settings
- >>> plugin selection menu: menu, shows available plugins, each can be instantiated onto the screen if a screen is selected, more about plugins and screens will follow further down below, plugin selection menu ui is defined by the active screen context area fluid ui settings
- >>> Manual: button, instantiates a manual plugin element onto the selected screen, planned but not yet implemented: showing the manual as a tooltip if no screen is selected
- >>> About: button, instantiates an about plugin element onto the selected screen, planned but not yet implemented: showing the about texts as a tooltip if no screen is selected
- >>> Shutdown: button, opens dialog and initiates hardware shutdown on confirmation
- >>> Off: button, opens dialog and initiates eboard shutdown on confirmation
- >>> Navigation Buttons: switch between screens
- >>> screen title: center bottom textbox shows the title of the screen context area
- >>> tooltip: showing fluid ui data text
- >>> context menu: right click menu, containing fluid ui menu and mainwindow context functions

→ **browser context:**

the eboard browser context area is located top left below the menu line if visible. It consists of a panel on the left, that gives an overview over existing screens, two buttons in the center, that allow hiding or showing the left or right area of the browser element and the right area, that is for details about the active screen and shows some data. A tooltip in the bottom half of the right area shows some statistics about the active screen. Changes to its fluid ui will affect all its controls except for

the representations of the screens, these will use the fluid ui data of the screen they represent. It contains the following controls:

- >>> Panel: shows screen representations in a panel within a scrollviewer
- >>> Add Eboard: button, adds a new screen to the panel with the properties in the input field
- >>> Edit Eboard: button, change the properties in the input field for the active screen
- >>> Delete Eboard: button, shows dialog and deletes active screen after confirmation {BROKEN}
- >>> Input field: Textblocks and TextBoxes between the buttons for name, depth, width and height of screen
- >>> data board: textblocks below delete eboard button show screen panel list position and fluid ui data title and text
- >>> data board tooltip: tooltip shows statistics(creation date and plugin count) of active screen, visible only above data board
- >>> center buttons: two buttons, left button will hide or show panel, right button will hide or show the buttons, input field and data board
- >>> tooltip: showing fluid ui data text
- >>> context menu: right click menu, containing fluid ui menu and eboard browser context functions

→ **screen context:**

the screen context area is located in the center of the mainwindow context area. Screens encapsulate empty virtual space and can be of any size and depth. Changes to its fluid ui will affect its empty space, its border, context menu, the mainwindow plugin selection, and the screen control ui. Screens are at the moment the only valid context area target for element instantiation, it is planned to explore docking panel for context areas in the mainwindow context, especially elements. For now, elements can be placed anywhere within the screens area and even beyond, if the user chooses to do so by entering coordinates into the elements fluid ui. Screen Control ui targets the active screen and has options to manipulate its elements. Screen context contains the following controls:

- >>> Panel: for its element collection
- >>> tooltip: showing fluid ui data text
- >>> context menu: right click menu, containing fluid ui menu and screen context functions

→ **element context:**

element context areas are encapsulated by a screen context area. Each element can be moved within its encapsulating context via left click and drag. Each element has a z property for its depth position on the screen and can be rotated. For now, elements get a z value of 1000 if they are moved, meaning your mouse cursor will hit any element with a higher z than that. It is possible to select elements. Selected elements can be manipulated by using the Screen Control ui or via an elements right click context menu. During the most recent refactoring some if not all right click context menu group functions like group placement, group rotation and so on have been disabled until there is time to adapt their logic to the new underlying system. Selection movement also got buggy during the process. An element context area contains the following controls:

- >>> plugin: control, encapsulating a feature, a function or a complex software
- >>> tooltip: showing fluid ui data title and text, as well as screen position and plugin type
- >>> context menu: right click menu, containing fluid ui menu and element context functions

→ **plugins:**

plugins are encapsulated by element context areas and can either use their elements design or, given the WPF framework underneath, have their own. Plugin authors can choose whether a plugin context menu is required or not. Same holds true for tooltips. It is planned but not yet implemented to use a property in the plugin for the plugins context menu. It is also planned but not yet implemented to restrict the plugins access towards eboard during runtime to simple get functions or properties and a limited amount of setting functions or properties. Each plugin outside of eboardsdk library must use the same base viewmodel to ensure it can be detected by and processed within the

eboard prototype. Each plugin must provide a data template for its views and viewmodels in the resource dictionary that is referenced in the viewmodel inheriting pluginbaseviewmodel. All dll-files within ..Eboard\Plugins\ are searched for the appropriate base type. It is quite easy to add plugins to the existing prototype. Right now plugins can be installed by copying a proper dll file into the Plugins folder that is below the Eboard folder in the directory that contains eboard.exe. Eboard only reads from the folder at program start. It is planned to improve the current system, so that plugins can be installed and deleted or deactivated with greater comfort during program runtime, which was already tested successfully, but requires some planning and some changes. Plugins are grouped into categories. The definition for the categories is not layouted in detail yet. A plugin can contain a simple function or offer a small set of functions and features like linking files. It could also be complex and feature rich. For now, the two existing Addon category plugins serve as mockups for complex software. Dependent on the settings within the implementation deriving from the pluginbaseviewmodel, plugin instantiation can be restricted. Of all the values of the enum, only a few are processed right now. Uptime plugin and drives f.e. have a oneperscreen instantiation policy. Any plugin with an unconstrained instantiation policy can be instantiated an unlimited amount per screen. Each element holds its own instance of its plugin type main viewmodel. A plugin contains the plugin controls, whatever they might be. Plugins can make use of the save and load functionality of the pluginbaseviewmodel to store content. In order to do so, a json serialization compatible model class is required.

→ **existing plugin features:** standard text(for creating and editing small texts, saves and loads stf named json files), basicAV(for videos, audio files or displaying pictures), image(for linking an image), link(for linking files, directories or websites), areas(lines and rows of plugin types), summoner(direct plugin instantiation), shapes(basic shapes, path shapes, text shapes for drawing and design), uptime(clock, date and system runtime), coordinates(plugin and mouse screen coordinates), drives(hdd/sdd/usb/cd drive access), BudgetWatcher(simple budget management), protocol(simple diary or note), plugin manager(plugin management, later intended for installation and uninstallation of plugins), about(displays license, version and concept information), manual(displays help texts, later maybe pdfs, flow documents or html or web resources), mynote and soundmix are mockups of more complex iterations of basicAV and protocol, countdown timer(set up a countdown), the other plugins have no functionality besides being mockups for more plugins of any kind. Kraken element plugin serves as a development tool. Its task is to be an external plugin loaded during runtime and access everything possible within the sdk and the application in order to read and write stuff from or into eboard during runtime, as if it where an external developed plugin with bad intentions in mind. In its earliest implementation all it does is accessing some public properties and displaying the findings into a textbox, but it already showed vulnerabilites like undesired access to some properties. It will help to highlight quirks in the design that have to be fixed in upcoming updates

→ **selection:**

Each element can be selected via shift + left mouse click. A group of elements can be selected using the Screen Control ui by setting up the selection extent and hitting the select button. Title select option will search for any element context area with the given title. Specific will show all plugins and allow the selection of instances of a certain type. The other settings will select all elements or the given category, or help narrowing down the plugin amount. Deselection can be done by clicking anywhere within the mainwindow context area, given the underlying surface is not a button or menu or context menu. It is planned to integrate more selection and filtering features.

→ **permance:**

Each fluid ui context area and any plugins content of plugins using its save and load methods will be saved upon program exit and sometimes upon crash. Any successfully saved data will be loaded upon program start and for some features upon user choice. It is planned to offer customization for

eboard data storage locations. A savepath can be set using the right click menu of the mainwindow context area, but the feature was not tested after the most recent refactoring and development work. All fluid ui data is saved in a variety of *.edf files. All content is saved in *.ecf files. Both extensions hide a json file with a certain model underneath. Right now, eboard only saves texts, links or fluid ui data to harddrives and restores its state from those files. It is planned to improve the existing permanency infrastructure in the future. For fastest access to data linked by and presented with eboard, local storage should remain. In the future, backups could be written to a cloud or loaded from if local storage is corrupt or missing.

→ **customization:**

Customization can be achieved using fluid ui context menu for each context area. Fluid ui configurations can be saved to and loaded from harddrives. In a context areas right click fluid ui menu, in the Data section, choose the Tab 'Fluid UI' to setup saving or loading of a fluid ui context. Loaded contexts will be applied onto the element. Saving will save the elements fluid ui as file on the selected location. The system is a first draft, some bugs are already known and noted. For selections of elements it is possible to load or reset fluid ui contexts using Screen Control Fluid UI tab. The prototype is quite flexible regarding customization. Due to the fluid ui features, any plugin or context area can become a piece of art, a sleek ci or a mockup for planned stuff. The *.edf or *.fcf files can be comfortable edited with code editors, allowing for fast changes before starting eboard or loading fluid ui configuration files or from within eboard using the fluid ui context menu. Visual and Radial brushes are in development. Drawing features and more shapes are planned. If you develop your own plugins, you can customize eboard even further. Mainwindow controls can be hidden, Tooltips can be deactivated under FluidUI – Data – Title and Text. Each screen can be named and colored differently, as well as each element. In order to customize eboard to your project needs, set up some screens, give them a name and organize some files or images or texts or mediafiles according to your needs. FileLinkArea plugin offers a multiclick button that allows to open all linked files at once.

→ **mousewheel:**

with mousewheel you can change an elements z value. Ctrl + mousewheel changes an elements rotation.

→ **leftclick:**

on some controls that show text, ctrl+leftclick can be used to change the control, so that it allows for a change of the text. On link and image element, ctrl+leftclick can be used to reset links. Double left click on mainwindow context will trigger showing or hiding of all mainwindow controls except the active screen.

Be advised, it is very likely that iplugin interface will be target for renaming, deletion or addition of properties and methods in future updates. The concept is a research in progress. Until a versioning is in place and until final decisions for plugin architecture have been achieved, you will have to recompile your plugin dlls with the updated iplugin interface, if such changes occurred.